

Notes on Distributional Reinforcement Learning

Preliminaries

These notes follow the modern treatment of distributional reinforcement learning in [BDR23].

1 Linear Function Approximation

This section follows Chapter 9 of [BDR23]. Let $\mathcal{X}$ be a finite, possibly very large state space. At time t , the agent is in state $X_t = x$, selects $A_t = a$, receives R_t , and moves to X_{t+1} . A policy π maps states to distributions over actions and $\gamma \in [0, 1)$ is the discount factor. We first discuss linear approximation for the state-value function

$$V^\pi : \mathcal{X} \rightarrow \mathbb{R}.$$

This is the cleanest setting because the input is only the state. Starting from $X_0 = x$ and then following π , define the discounted return

$$G^\pi(x) = \sum_{t=0}^{\infty} \gamma^t R_t.$$

The state-value function is the mean return:

$$V^\pi(x) = \mathbb{E}_\pi[G^\pi(x)].$$

It is the fixed point of the Bellman operator $\mathcal{T}^\pi$, defined by

$$(\mathcal{T}^\pi V)(x) = \mathbb{E}_{A \sim \pi(\cdot|x)}[\mathbb{E}[R + \gamma V(X') \mid X = x, A]].$$

The Bellman equation is $V^\pi = \mathcal{T}^\pi V^\pi$.

1.1 Function Approximation and Aliasing

In a small state space, V^π can be stored as a table with one number per state. Function approximation instead shares parameters across states. This is necessary when $\mathcal{X}$ cannot be enumerated and it permits generalization to states not yet observed. The cost is *aliasing*: a shared parameter changes the prediction at several states. If two aliased states require different values, no parameter vector represents both exactly. With bootstrapping, the resulting error may propagate to states that are not themselves aliased.

Definition 1: State representation and linear value approximation

Let $n \in \mathbb{N}$. A state representation is a mapping

$$\phi : \mathcal{X} \rightarrow \mathbb{R}^n, \quad \phi(x) = \begin{pmatrix} \phi_1(x) \\ \vdots \\ \phi_n(x) \end{pmatrix}.$$

For $w \in \mathbb{R}^n$, the linear value function approximation is

$$V_w(x) = \phi(x)^\top w,$$

where each ϕ_i is a basis function and $\phi_i(x)$ is a feature.

The model is linear in its weights and

$$V_{\alpha w_1 + \beta w_2} = \alpha V_{w_1} + \beta V_{w_2}, \quad \nabla_w V_w(x) = \phi(x).$$

For finite $\mathcal{X}$, collect the row vectors $\phi(x)^\top$ in a feature matrix $\Phi \in \mathbb{R}^{|\mathcal{X}| \times n}$. Then

$$V_w = \Phi w, \quad \mathcal{F}_\phi = \{\Phi w : w \in \mathbb{R}^n\}.$$

For action values, either take $\phi : \mathcal{X} \times \mathcal{A} \rightarrow \mathbb{R}^n$ and $Q_w(x, a) = \phi(x, a)^\top w$, or use a common state representation with one weight vector per action, $Q_w(x, a) = \phi(x)^\top w_a$.

1.2 Optimal Approximation in a Weighted Norm

Let $\xi \in \mathcal{P}(\mathcal{X})$ express the relative importance of the states. Define

$$\|V - V'\|_{\xi,2} = \left(\sum_{x \in \mathcal{X}} \xi(x) (V(x) - V'(x))^2 \right)^{1/2}.$$

The best approximation to V^π solves

$$\min_{w \in \mathbb{R}^n} \|V^\pi - V_w\|_{\xi,2}.$$

Let $\Xi = \text{diag}(\xi(x) : x \in \mathcal{X})$. If ξ has full support and the columns of Φ are linearly independent, the unique solution is

$$w^* = (\Phi^\top \Xi \Phi)^{-1} \Phi^\top \Xi V^\pi.$$

Thus the state weighting ξ is part of the approximation problem: changing how often states are sampled generally changes the best attainable weights.

1.3 The Projected Bellman Operator

In general, $V \in \mathcal{F}_\phi$ does not imply $\mathcal{T}^\pi V \in \mathcal{F}_\phi$. Define the weighted orthogonal projection $\Pi_{\phi,\xi}$ by

$$(\Pi_{\phi,\xi} V)(x) = \phi(x)^\top w^*, \quad w^* \in \arg \min_{w \in \mathbb{R}^n} \|V - V_w\|_{\xi,2}.$$

Approximate dynamic programming iterates

$$V_{k+1} = \Pi_{\phi, \xi} \mathcal{T}^\pi V_k.$$

Projection removes the component of the Bellman update that the features cannot represent. Repeated projection can compound approximation error.

Let ξ_π be a fully supported steady-state distribution under π . Then P^π and Π_{ϕ, ξ_π} are nonexpansions in the ξ_π -weighted L_2 norm. Consequently,

$$\|\Pi_{\phi, \xi_\pi} \mathcal{T}^\pi V - \Pi_{\phi, \xi_\pi} \mathcal{T}^\pi V'\|_{\xi_\pi, 2} \leq \gamma \|V - V'\|_{\xi_\pi, 2}.$$

The projected operator has a unique fixed point

$$\hat{V}^\pi = \Pi_{\phi, \xi_\pi} \mathcal{T}^\pi \hat{V}^\pi,$$

and the book's sharper orthogonal-projection argument gives

$$\|\hat{V}^\pi - V^\pi\|_{\xi_\pi, 2} \leq \frac{1}{\sqrt{1 - \gamma^2}} \|\Pi_{\phi, \xi_\pi} V^\pi - V^\pi\|_{\xi_\pi, 2}.$$

The fixed point $\hat{V}^\pi$ is generally not the direct best approximation $\Pi_{\phi, \xi_\pi} V^\pi$.

1.4 Monte Carlo and Semi-Gradient TD

For independent state–return samples (x_k, g_k) , stochastic gradient descent on $(g_k - \phi(x_k)^\top w)^2$ gives

$$w \leftarrow w + \alpha_k (g_k - \phi(x_k)^\top w) \phi(x_k).$$

For a sampled transition (x_k, a_k, r_k, x'_k) , replace g_k by a bootstrapped target. The TD error is

$$\delta_k = r_k + \gamma \phi(x'_k)^\top w - \phi(x_k)^\top w,$$

and the semi-gradient update is

$$w \leftarrow w + \alpha_k \delta_k \phi(x_k).$$

It is called a semi-gradient because the target is held fixed during differentiation. The gradient of the full squared Bellman error would contain an additional term involving $\phi(x'_k)$. The connection to the projected Bellman operator is clearest with separate target weights $\tilde{w}$:

$$w \leftarrow w + \alpha_k (r_k + \gamma \phi(x'_k)^\top \tilde{w} - \phi(x_k)^\top w) \phi(x_k).$$

For fixed $\tilde{w}$, this is SGD toward the solution

$$\Phi w^* = \Pi_{\phi, \xi} \mathcal{T}^\pi V_{\tilde{w}}.$$

Setting $\tilde{w} = w$ creates a moving-target feedback loop. Even when the exact projected operator converges, semi-gradient iterates can diverge under an unsuitable sampling distribution (Baird's counterexample). On-policy sampling according to the steady-state distribution

restores classical convergence guarantees under the usual feature and step-size assumptions.

2 Distributional Value Approximation

Distributional reinforcement learning replaces the scalar value function by a return-distribution function. Instead of storing only the mean $Q^\pi(x, a) = \mathbb{E}[G^\pi(x, a)]$, we keep the law of the return:

$$\eta^\pi(x, a) = \text{Law}(G^\pi(x, a)).$$

Thus

$$\eta^\pi : \mathcal{X} \times \mathcal{A} \rightarrow \mathcal{P}(\mathbb{R}),$$

where $\mathcal{P}(\mathbb{R})$ denotes probability distributions on the real line. The classical action-value function is recovered by taking expectations:

$$Q^\pi(x, a) = \int_{\mathbb{R}} z \eta^\pi(x, a)(dz).$$

The distributional Bellman operator is most cleanly written using pushforwards. For $r \in \mathbb{R}$, define the affine map

$$f_{r,\gamma}(z) = r + \gamma z.$$

If $\nu \in \mathcal{P}(\mathbb{R})$, then $(f_{r,\gamma})_\# \nu$ is the distribution of $r + \gamma Z$ when $Z \sim \nu$. The distributional Bellman operator $\mathcal{T}^\pi$ maps a return-distribution function η to

$$(\mathcal{T}^\pi \eta)(x, a) = \mathbb{E}[(f_{R,\gamma})_\# \eta(X', A') \mid X = x, A = a], \quad A' \sim \pi(\cdot \mid X').$$

The expectation averages over the random reward, the next state, and the next action sampled from the policy. The distributional Bellman equation is

$$\eta^\pi = \mathcal{T}^\pi \eta^\pi.$$

Taking means on both sides recovers the usual Bellman equation for Q^π , but the distributional equation is stronger: it describes the whole random return, not only its expectation. There is an important distinction between the two approximation mechanisms now present. A probability representation approximates the *output* at one state by finitely many numbers. Function approximation shares those numbers *across states*. Moreover, probability distributions do not form a vector space: in general it is meaningless to require

$$\eta_{w_1+w_2}(x) = \eta_{w_1}(x) + \eta_{w_2}(x),$$

because the right side has total mass two. Following Chapter 9, “linear distributional approximation” therefore means that the parameters of a valid probability representation depend linearly on $\phi(x)$.

Just as linear TD restricts Q to a class $\mathcal{Q}_\phi$, distributional methods restrict η to a class of approximate distributions. The quantile family is one of the most important such choices.

For a distribution ν with cumulative distribution function F_ν , define its quantile function by

$$F_\nu^{-1}(\tau) = \inf\{z \in \mathbb{R} : F_\nu(z) \geq \tau\}, \quad \tau \in (0, 1).$$

Choose N quantile levels, usually the midpoints

$$\tau_i = \frac{2i-1}{2N}, \quad i = 1, \dots, N.$$

An N -quantile approximation represents $\eta(x, a)$ by the empirical distribution

$$\eta_\theta(x, a) = \frac{1}{N} \sum_{i=1}^N \delta_{\theta_i(x, a)},$$

where δ_z is a Dirac mass at z . The parameter $\theta_i(x, a)$ is interpreted as the predicted τ_i -quantile of the return distribution at (x, a) .

The direct distributional analogue of linear action-value approximation is to make each quantile location linear in the same feature vector:

$$\theta_i(x, a) = \phi(x, a)^\top w_i, \quad w_i \in \mathbb{R}^d.$$

Equivalently, with $W = (w_1, \dots, w_N) \in \mathbb{R}^{d \times N}$,

$$\eta_W(x, a) = \frac{1}{N} \sum_{i=1}^N \delta_{\phi(x, a)^\top w_i}.$$

The scalar approximation $Q_w(x, a)$ has become a measure-valued approximation $\eta_W(x, a)$. The same features are now asked to explain not only the average return, but also the spread, asymmetry, and tail behavior of the return.

3 Quantile TD

Quantile TD applies temporal-difference learning to the quantile parameterization. For policy evaluation, sample a transition and next action

$$(x_t, a_t, r_t, x'_t, a'_t), \quad a'_t \sim \pi(\cdot \mid x'_t).$$

The distributional Bellman target is represented by the N target atoms

$$y_j = r_t + \gamma \theta_j(x'_t, a'_t), \quad j = 1, \dots, N.$$

The current prediction at (x_t, a_t) has atoms

$$z_i = \theta_i(x_t, a_t), \quad i = 1, \dots, N.$$

Quantile regression uses the asymmetric absolute loss

$$\rho_\tau(u) = u(\tau - \mathbb{1}_{\{u < 0\}}).$$

The empirical Quantile TD loss is then

$$L(W) = \frac{1}{N} \sum_{i=1}^N \sum_{j=1}^N \rho_{\tau_i}(y_j - z_i).$$

The residual $y_j - z_i$ compares an atom of the Bellman target distribution with the predicted τ_i -quantile at the current state-action pair. Minimizing this loss moves $\eta_W(x_t, a_t)$ toward

$$\frac{1}{N} \sum_{j=1}^N \delta_{r_t + \gamma \theta_j(x'_t, a'_t)} = (f_{r_t, \gamma})_{\#} \eta_W(x'_t, a'_t).$$

For linear quantiles, a subgradient update for each head is

$$w_{i,t+1} = w_{i,t} + \alpha_t \left(\frac{1}{N} \sum_{j=1}^N (\tau_i - \mathbb{1}_{\{y_j < z_i\}}) \right) \phi(x_t, a_t), \quad i = 1, \dots, N.$$

This is the exact analogue of the semi-gradient TD update:

$$\delta_t = r_t + \gamma Q_{w_t}(x'_t, a'_t) - Q_{w_t}(x_t, a_t) \rightsquigarrow \{y_j - z_i\}_{i,j=1}^N.$$

The scalar TD target is replaced by a Bellman target distribution, and the single scalar TD error is replaced by all pairwise quantile residuals.

In implementations, one often replaces ρ_{τ} by a quantile Huber loss. This is the smooth loss used in QR-DQN [Dab+18a]. Define

$$\mathcal{L}_{\kappa}(u) = \begin{cases} \frac{1}{2}u^2, & |u| \leq \kappa, \\ \kappa \left(|u| - \frac{1}{2}\kappa \right), & |u| > \kappa, \end{cases}$$

and

$$\rho_{\tau}^{\kappa}(u) = \left| \tau - \mathbb{1}_{\{u < 0\}} \right| \frac{\mathcal{L}_{\kappa}(u)}{\kappa}.$$

The corresponding loss is

$$L_{\kappa}(W) = \frac{1}{N} \sum_{i=1}^N \sum_{j=1}^N \rho_{\tau_i}^{\kappa}(y_j - z_i).$$

The Huber version keeps the asymmetric quantile penalty but smooths the non-differentiability around zero, which is useful when the quantile locations are produced by a neural network. For control, the next action is usually selected greedily with respect to the mean of the predicted distribution:

$$a^*(x') \in \arg \max_{a' \in \mathcal{A}} \frac{1}{N} \sum_{j=1}^N \theta_j(x', a').$$

The target atoms become

$$y_j = r + \gamma \theta_j(x', a^*(x')), \quad j = 1, \dots, N.$$

This is the quantile analogue of replacing policy evaluation TD by Q-learning.

4 Linear Categorical TD

Quantile locations are one possible linearly parameterized representation. The categorical alternative fixes locations

$$\theta_1 < \dots < \theta_m$$

and makes the logits of their probabilities linear in the state features. For weights $w_1, \dots, w_m \in \mathbb{R}^n$, define

$$p_i(x; w) = \frac{\exp(\phi(x)^\top w_i)}{\sum_{j=1}^m \exp(\phi(x)^\top w_j)}, \quad \eta_w(x) = \sum_{i=1}^m p_i(x; w) \delta_{\theta_i}.$$

The softmax is not incidental: arbitrary linear coefficients would not necessarily be nonnegative or sum to one.

For a sample transition (x, a, r, x') , apply the affine Bellman map and then the categorical projection:

$$\bar{\eta}(x) = \Pi_c(f_{r,\gamma})_{\#} \eta_w(x') = \sum_{i=1}^m \bar{p}_i \delta_{\theta_i}.$$

Equivalently, if $\zeta_{i,j}(r)$ denotes the fraction of the mass originally at θ_j assigned to θ_i by the projection, then

$$\bar{p}_i = \sum_{j=1}^m \zeta_{i,j}(r) p_j(x'; w).$$

Linear CTD minimizes the cross-entropy

$$L(w) = - \sum_{i=1}^m \bar{p}_i \log p_i(x; w).$$

The softmax-cross-entropy gradient gives the particularly simple update

$$w_i \leftarrow w_i + \alpha \underbrace{(\bar{p}_i - p_i(x; w))}_{\text{categorical TD error}} \phi(x).$$

Scalar TD, linear QTD, and linear CTD therefore share the template

$$w_i \leftarrow w_i + \alpha \varepsilon_i \phi(x),$$

but use different error signals. The scalar TD error is unbounded. The QTD and CTD errors lie in $[-1, 1]$, which suggests different learning dynamics even before a nonlinear network is introduced. Both distributional rules are semi-gradients: the Bellman target is treated as fixed when differentiating.

5 Deep Neural Networks and DQN

Chapter 10 of [BDR23] replaces the fixed representation ϕ by a representation learned jointly with the prediction heads. In the canonical DQN architecture, four preprocessed 84×84

Atari frames are passed through three convolutional layers and a fully connected layer. The latter produces a 512-dimensional feature vector; a final linear layer outputs one action value $Q_w(x, a)$ for each $a \in \mathcal{A}$. Thus the penultimate layer may still be viewed as a state representation, but one whose parameters are learned.

DQN behaves ε -greedily with respect to Q_w and stores transitions in a replay buffer. Given online weights w , target weights $\tilde{w}$, and a sampled transition (x, a, r, x') , its target is

$$y = r + \gamma \max_{a' \in \mathcal{A}} Q_{\tilde{w}}(x', a').$$

The squared loss and semi-gradient update are

$$L(w) = (y - Q_w(x, a))^2, \quad w \leftarrow w + \alpha(y - Q_w(x, a)) \nabla_w Q_w(x, a).$$

When $Q_w(x, a) = \phi(x, a)^\top w$, the gradient is exactly $\phi(x, a)$ and the linear update is recovered. In the nonlinear case, backpropagation efficiently computes the gradient through all layers. Replay reduces temporal correlations and reuses data; the frozen target weights slow the moving-target feedback loop. The Huber loss is often substituted for the squared loss to limit the influence of large TD errors.

6 Gateway to Deep Representations

The linear approximation above separates representation and prediction:

fixed features $\phi(x, a)$ and linear heads w_i .

This is the useful gateway into deep distributional reinforcement learning. The deep version keeps the same conceptual decomposition, but replaces the fixed feature map by a learned representation. A neural network maps the observation or state to a feature vector

$$\psi_\xi(x) \in \mathbb{R}^d,$$

and the final layers produce a distributional prediction. In the quantile case one may write

$$\theta_i(x, a; \omega) = f_i(\psi_\xi(x), a), \quad i = 1, \dots, N.$$

The return-distribution approximation becomes

$$\eta_\omega(x, a) = \frac{1}{N} \sum_{i=1}^N \delta_{\theta_i(x, a; \omega)}.$$

The parameter ω includes both the representation parameters ξ and the prediction heads. Linear Quantile TD is recovered as the special case in which ψ_ξ is fixed and f_i is linear. Deep quantile methods instead optimize all of ω by backpropagating through the distributional TD loss.

More generally, a deep distributional network outputs an $|\mathcal{A}| \times m$ matrix of distribution parameters. C51 interprets each action's row as categorical logits, QR-DQN as quantile locations, and IQN makes the quantile level an additional network input. The probability representation changes, but online/target networks, replay, semi-gradient targets, and ε -greedy behavior remain the common scaffold.

7 C51

C51 fixes support locations $\theta_1 < \dots < \theta_m$ and lets the network output logits $\varphi_i(x, a; w)$. They define

$$p_i(x, a; w) = \frac{e^{\varphi_i(x, a; w)}}{\sum_{j=1}^m e^{\varphi_j(x, a; w)}}, \quad \eta_w(x, a) = \sum_{i=1}^m p_i(x, a; w) \delta_{\theta_i}.$$

The induced action value and greedy target-network action are

$$Q_w(x, a) = \sum_{i=1}^m p_i(x, a; w) \theta_i, \quad a_{\tilde{w}}(x') \in \arg \max_{a'} Q_{\tilde{w}}(x', a').$$

For a transition (x, a, r, x') , form the projected Bellman target

$$\bar{\eta}(x, a) = \Pi_c(f_{r, \gamma})_{\#} \eta_{\tilde{w}}(x', a_{\tilde{w}}(x')) = \sum_{j=1}^m \bar{p}_j \delta_{\theta_j},$$

and minimize

$$L(w) = - \sum_{j=1}^m \bar{p}_j \log p_j(x, a; w).$$

This is the nonlinear counterpart of linear CTD. Unlike the linear case, weights in early layers affect every output probability, so there is no independent per-particle update. The original algorithm uses $m = 51$ evenly spaced locations. The support endpoints matter: projected mass outside the support is clipped to an endpoint, introducing representation bias.

8 QR-DQN

QR-DQN is the deep control version of Quantile TD [Dab+18a]. It uses a neural network to output N quantile locations for each action:

$$\theta_i(x, a; \omega), \quad i = 1, \dots, N, \quad a \in \mathcal{A}.$$

For each state-action pair, the predicted return distribution is

$$\eta_{\omega}(x, a) = \frac{1}{N} \sum_{i=1}^N \delta_{\theta_i(x, a; \omega)}.$$

The predicted scalar action-value is the mean of this empirical distribution:

$$Q_{\omega}(x, a) = \frac{1}{N} \sum_{i=1}^N \theta_i(x, a; \omega).$$

Thus QR-DQN can still choose greedy actions in the usual value-based way, but the value used for action selection is obtained by averaging the learned quantiles.

As in DQN, QR-DQN uses two sets of parameters. The online network has parameters ω and is updated by gradient descent. The target network has parameters $\bar{\omega}$ and is held fixed

for several updates before being copied from ω . For a transition

$$(x, a, r, x', d),$$

where $d \in \{0, 1\}$ indicates termination, the greedy next action is selected by the mean of the target-network distribution:

$$a^*(x') \in \arg \max_{a' \in \mathcal{A}} \frac{1}{N} \sum_{j=1}^N \theta_j(x', a'; \bar{\omega}).$$

The target quantile atoms are

$$y_j = r + \gamma(1 - d)\theta_j(x', a^*(x'); \bar{\omega}), \quad j = 1, \dots, N.$$

The online prediction atoms for the sampled action are

$$z_i = \theta_i(x, a; \omega), \quad i = 1, \dots, N.$$

QR-DQN minimizes the pairwise quantile Huber loss

$$L_{\text{QR}}(\omega) = \frac{1}{N} \sum_{i=1}^N \sum_{j=1}^N \rho_{\tau_i}^{\kappa}(y_j - z_i),$$

with quantile levels

$$\tau_i = \frac{2i - 1}{2N}.$$

In a minibatch $\mathcal{B}$ sampled from replay memory, the objective is

$$\mathcal{L}_{\text{QR}}(\omega) = \frac{1}{|\mathcal{B}|} \sum_{(x, a, r, x', d) \in \mathcal{B}} L_{\text{QR}}(\omega; x, a, r, x', d).$$

The target atoms are treated as constants when differentiating with respect to ω ; this is the usual semi-gradient idea inherited from TD learning and DQN.

The quantile Huber loss is important because the raw quantile loss $\rho_{\tau}(u)$ is non-smooth at zero. QR-DQN uses

$$\mathcal{L}_{\kappa}(u) = \begin{cases} \frac{1}{2}u^2, & |u| \leq \kappa, \\ \kappa(|u| - \frac{1}{2}\kappa), & |u| > \kappa, \end{cases}$$

and

$$\rho_{\tau}^{\kappa}(u) = \left| \tau - \mathbb{1}_{\{u < 0\}} \right| \frac{\mathcal{L}_{\kappa}(u)}{\kappa}.$$

For small residuals, the loss has a squared-error shape, which gives smoother gradients. For large residuals, it grows linearly, which prevents very large TD errors from dominating the update. The factor $|\tau - \mathbb{1}_{\{u < 0\}}|$ keeps the asymmetry needed for quantile regression: underestimation and overestimation are penalized differently depending on the quantile level τ .

How QR-DQN Relates to DQN

DQN learns one scalar per action:

$$x \mapsto \{Q_\omega(x, a) : a \in \mathcal{A}\}.$$

QR-DQN learns a collection of quantiles per action:

$$x \mapsto \{\theta_i(x, a; \omega) : i = 1, \dots, N, a \in \mathcal{A}\}.$$

The scalar Bellman target

$$r + \gamma \max_{a'} Q_{\bar{\omega}}(x', a')$$

is replaced by the empirical target distribution

$$\frac{1}{N} \sum_{j=1}^N \delta_{r+\gamma(1-d)\theta_j(x', a^*(x'); \bar{\omega})}.$$

The squared TD loss is replaced by the pairwise quantile Huber loss. Action selection remains risk-neutral unless we deliberately change it: QR-DQN chooses actions by the mean of the predicted quantiles. The learned object, however, is much richer than the mean.

Representation Learning Signal

The representation $\psi_\xi(x)$ in QR-DQN is trained by many losses at once. For a single transition, the network sees all residuals

$$y_j - \theta_i(x, a; \omega), \quad i, j = 1, \dots, N.$$

Lower quantile heads receive large gradients when the lower tail is poorly placed; upper quantile heads receive large gradients when the upper tail is poorly placed. Thus the shared representation is shaped not only by expected return, but by the whole geometry of the target return distribution.

This gives a useful interpretation of the empirical strength of distributional methods. A scalar DQN representation only needs to make the conditional mean easy to predict. A QR-DQN representation must make many parts of the return distribution easy to predict simultaneously. The extra prediction problem can act as a richer training signal, even when the final policy is still selected by the mean.

9 Beyond Fixed Quantile Locations

QR-DQN fixes the quantile levels $\tau_1, \dots, \tau_N$. Implicit Quantile Networks (IQN) generalize this idea by learning a function of the quantile level itself [Dab+18b]. Instead of outputting only a finite list of quantile locations, IQN models

$$(x, a, \tau) \mapsto Z_\omega(x, a; \tau), \quad \tau \in (0, 1).$$

During training, quantile levels are sampled, for example

$$\tau_i, \tau'_j \sim \mathcal{U}(0, 1),$$

and the same quantile Huber principle is applied to residuals

$$r + \gamma(1 - d)Z_{\bar{\omega}}(x', a^*; \tau'_j) - Z_{\omega}(x, a; \tau_i).$$

This makes the quantile function itself part of the model. The network must learn a representation of both the state and the quantile level. Conceptually, IQN moves from a finite set of quantile heads to a continuous return-distribution model.

This is the natural gateway to deep representations. Chapter-level questions about deep distributional RL can be phrased as:

What representation makes $\eta^\pi(x, a)$ easy to approximate?

Linear Quantile TD answers this with fixed ϕ and linear quantile heads. QR-DQN replaces ϕ by a learned deep representation and keeps finitely many quantile heads. IQN goes one step further and makes the quantile level an input, so that the representation must support an entire quantile function. In all cases, the core idea is the same: approximate the distributional Bellman fixed point, not merely its expectation.

References

- [BDR23] Marc G. Bellemare, Will Dabney, and Mark Rowland. *Distributional Reinforcement Learning*. MIT Press, 2023.
- [Dab+18a] Will Dabney et al. “Distributional Reinforcement Learning with Quantile Regression”. In: *Proceedings of the AAAI Conference on Artificial Intelligence*. 2018.
- [Dab+18b] Will Dabney et al. “Implicit Quantile Networks for Distributional Reinforcement Learning”. In: *Proceedings of the 35th International Conference on Machine Learning*. 2018, pp. 1096–1105.